

スマートウォッチとヘルスコネクト連携：技術・実装ガイドライン

本ガイドラインは、Android プラットフォームにおける健康データ管理の中核を担う「ヘルスコネクト（Health Connect）」と、主要ウェアラブルデバイスの統合に関する技術仕様をまとめたものである。ヘルスケア IT・テクニカルソリューションアーキテクトの視点から、データの断片化解消とユーザー体験の最大化を目指した実装アプローチを提示する。

1. ヘルスコネクト（Health Connect）の概要と役割

ヘルスコネクトは、Android デバイス上で異なるアプリ間のデータ連携を支える統合データプラットフォームである。従来、各アプリが個別のウェアラブルベンダーと「1 対 1」の API 統合を行う必要があったが、ヘルスコネクトの導入により、標準化されたスキーマを介した一元的なデータアクセスが可能となった。

基本概念とデータアーキテクチャ

- デバイス中心のデータ保存:** データはクラウドではなく、ユーザーのデバイス上に暗号化された状態で保存される。これにより、低遅延なアクセスと強固なデータプライバシーを両立している。
- スキーマの標準化:** 50 以上のデータカテゴリに対して共通のデータ構造を提供し、アプリ間の相互運用性を担保する。
- データ主権の確立:** ユーザーは「どのアプリが」「どのデータに」アクセスするかを項目単位で詳細に制御でき、情報の透明性が確保されている。

Google Fit API からの移行計画

既存の Google Fit API は、2026 年末をもってサポートが終了する。開発者は、deprecation（非推奨化）に伴うリスクを避けるため、早期の移行が推奨される。

- 2026 年末まで:** 現行の Google Fit API のサポート継続。
- 移行パス:** ヘルスコネクト SDK への統合を完了させ、データソースの移行ガイドラインに従う必要がある。
- アーキテクチャの変更:** アカウント中心の Fit API から、デバイス中心のヘルスコネクトへ設計思想を転換することが必須となる。

OS バージョン別の提供形態

Android OS のバージョンにより、ヘルスコネクトのシステムへの組み込み形態が異なる点に注意が必要である。

項目	Android 14 以降	Android 13 以前
提供形態	Android フレームワークの一部（モジュール）	APK（Google Play ストアから提供）
更新方法	Google Play システムアップデート経由	アプリストア経由の APK 更新
セットアップ	設定画面に標準搭載、別途インストール不要	ユーザーによる APK のインストールが必須

2. スマートウォッチの対応状況とデータ連携

主要ウェアラブルブランドは、専用のモバイルアプリをブリッジとしてヘルスコネクと連携している。アーキテクトとしては、単なるデータ同期に留まらず、クロスプラットフォームにおける OS 要件（例：2026 年版 Garmin Connect アプリの **iOS 18.2 以上 必須化**）を考慮した設計が求められる。

ガーミン（Garmin）のデータ統合

「Garmin Connect」アプリをハブとして、高精度なバイタルおよび活動データをプラットフォームへ供給する。取得可能な主要データと活用シーンは以下の通り。

- **心拍数・心拍変動 (HRV):** 安静時心拍の傾向監視による体調変化の早期検知。
- **VO2max（最大酸素摂取量）:** 長期的な持久力向上とトレーニング強度の最適化。
- **リカバリーアドバイザー:** 運動負荷と睡眠の質に基づく「休息の数値化」による故障防止。
- **睡眠ステージ (REM 含む):** 深い睡眠、浅い睡眠の配分を通じた生活リズムの改善。
- **マルチバンド GPS 位置情報:** 都市部や山間部での高精度な移動軌跡の記録。

シチズン（Citizen）「リィイバー」の IoT 拡張性

シチズンの「Eco-Drive Riiiver」は、独自の IoT プラットフォーム「iiidea」を用いることで、ヘルスケアの枠を超えた拡張を可能にしている。

- **iiidea による機能拡張:** ユーザーが特定の機能を時計のボタン（物理ボタン）に割り当て可能。
- **ヘルスケアへの応用:** 「iiidea」をトリガーとして、服薬ログの記録や特定の症状のタイムスタンプ保存など、標準的なアクティビティトラッキング以外の健康管理ワークフローを構築できる。
- **外部サービス連携:** IoT 機器と連動し、心拍数や活動量に基づいたスマートホーム制御（例：帰宅時のライト点灯）など、QOL 向上に寄与する。

3. アプリ間データ活用とシステム体験の向上

サポートデータカテゴリの拡充

ヘルスコネクは、以下の主要項目を含む広範なデータ型をサポートしている。特に、近年需要が高まっている **栄養** や **月経周期** データの統合は、包括的な健康管理アプリにおいて不可欠である。

- **アクティビティ:** 歩数、消費カロリー、距離、エクササイズセッション。
- **睡眠:** 睡眠時間、睡眠ステージ（REM、深い睡眠、浅い睡眠）。
- **身体測定値:** 体重、体脂肪率、身長、BMI。
- **バイタル:** 心拍数、血圧、体温、血中酸素ウェルネス。
- **栄養:** カロリー摂取量、マクロ栄養素（タンパク質、炭水化物、脂質等）、水分補給。
- **サイクル記録:** 月経周期、排卵関連の指標。

UI/UX の最適化提言

ウェアラブルデバイスの UI 設計において、アーキテクトは情報の即時性とプライバシー保護の両立を考慮すべきである。

1. **ウィジェットとコンプリケーション:** アプリを起動せずに、現在の心拍数や活動達成度を提示する。ただし、ロック画面上のコンプリケーションにおいては、**バイタルデータなどの機密情報を自動的にマスキング（非表示）にする** 設定を標準とすべきである。

2. **アクションボタンの活用:** Apple Watch Ultra 等に搭載されるアクションボタンには、最も頻度の高いアクションを割り当てる。
 - **ラベルルール:** ユーザー設定画面では「**3 単語以内**」で「**動詞から始まる**」簡潔なラベル（例：「Start Race」、「Log Water」）を使用する。
3. **コンテキストの維持:** iOS のライブアクティビティのような軽量のマルチタスク機能を活用し、現在のトレーニングセッションを中断することなく、他の操作を可能にする設計が望ましい。

4. Android アプリでのデータ読み込み：実装手順

新規 Android アプリにおいてヘルスコネクトを統合するための、論理的な開発ステップを以下に示す。

1. **SDK の追加** `build.gradle` (Module level) にヘルスコネクトの依存関係を追加する。
2. **可視性の宣言** `AndroidManifest.xml` の `<queries>` タグ内にヘルスコネクトのパッケージを宣言する。
3. **権限の宣言** 必要なデータ型に対応する権限を宣言する。
4. **プライバシーポリシーのインテントフィルタ設定** ユーザーへの説明責任を果たすため、権限ダイアログから遷移可能なプライバシーポリシー画面を設定する。インテントアクションには `androidx.health.ACTION_SHOW_PERMISSIONS_RATIONALE` を明示的に指定する必要がある。

5. 具体的なコード例と技術仕様

クライアントインスタンスの取得と権限リクエスト

Kotlin を用いた実装例を示す。StepsRecord や HeartRateRecord は `androidx.health.connect.client.records` パッケージを参照する。

```
// ヘルスコネクトクライアントの取得
val healthConnectClient = HealthConnectClient.getOrCreate(context)

// 権限セットの定義
val permissions = setOf(
    HealthPermission.getReadPermission(StepsRecord::class),
    HealthPermission.getWritePermission(StepsRecord::class),
    HealthPermission.getReadPermission(HeartRateRecord::class)
)

// 権限リクエストの実行
val requestPermissionActivityContract =
    PermissionController.createRequestPermissionResultContract()
val requestPermissions = registerForActivityResult(requestPermissionActivityContract)
{ granted ->
    if (granted.containsAll(permissions)) {
        // 全権限承認時のロジック
    }
}

fun checkAndRequestPermissions() {
    lifecycleScope.launch {
        val granted = healthConnectClient.permissionController.getGrantedPermissions()
        if (!granted.containsAll(permissions)) {
            requestPermissions.launch(permissions)
        }
    }
}
```

}

開発上の重要事項

- **リクエスト拒否時のロックアウト:** ユーザーが権限リクエストを3回以上拒否した場合、OSレベルでアプリがロックアウトされる。以降、システムダイアログは自動的に抑制されるため、アプリ側で「設定画面への誘導」を行うUIを実装しておく必要がある。

6. まとめと今後の展望

スマートウォッチとヘルスコネクットの連携は、2026年に向けてさらなる進化を遂げる。

- **AI 画像認識による栄養管理:** Garmin Connect+等で見られる、AI 画像認識を用いた食事トラッキング機能の拡充により、活動量と摂取栄養素を相関させた高度な分析が可能になる。
- **データの優先順位付け:** 複数デバイスを所有するユーザーに対し、ヘルスコネクット側で優先度の高いデータソースを選択できる仕組みが、エコシステム全体の信頼性を担保する。
- **核心的価値:** ヘルスコネクットの本質は、ユーザーによる **詳細なプライバシー制御** と、特定のプラットフォームに縛られない **データの透明性** にある。ソリューションアーキテクトは、このデータ主権を尊重しつつ、統合されたデータを価値ある「洞察」へと変換する実装に注力すべきである。